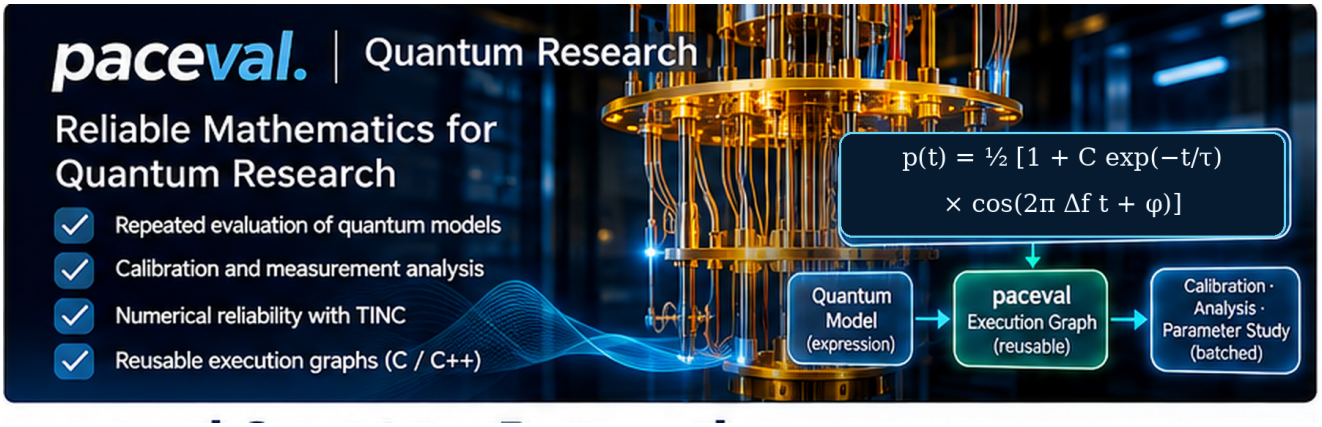
IMPLEMENTED RESEARCH DEMONSTRATOR | NOVEMBER 2026

Reliable Mathematics for Quantum Research

A verified C++ implementation of Ramsey calibration mathematics and explicit two-qubit circuit expressions using reusable paceval execution graphs.



Outcome: The selected use cases were implemented and executed with the original paceval Linux runtime, checked with independent C++ reference calculations, and assessed with additional high-precision numerical checks. All tests described in this report use controlled synthetic data.

Actual implementation	Independent verification	Scientific boundary
Five reusable expression graphs plus **two** TINC graphs	**24 / 24** sampled TINC point intervals contained independent 110-digit reference values	Classical mathematics for quantum research; no results obtained from a quantum processor
Combined Ramsey evaluation and five detuning-frequency hypotheses	Analytic identities, frequency derivative and a four-amplitude circuit statevector checked	No complete device calibration fit or generic quantum-circuit simulation claim

The purpose of this publication is to document **successful technical feasibility** and provide an independently reproducible demonstration, and to state exactly which scientific calculations and checks were implemented.

1 Scientific objective and verified scope

Many quantum-research workflows involve repeated evaluation of classical mathematical expressions: calibration response functions, likelihood contributions, numerical derivatives, parameterized expectation values and checks on computed numerical results. The present study establishes that these selected expressions can be represented as persistent paceval computations and evaluated for changing input values.

The research is deliberately scoped to **two analytically defined mathematical examples**. A Ramsey experiment provides a compact model for calibration studies. A two-qubit circuit with a specified observable provides an exact and independently checkable expectation-value expression. Both are evaluated in classical software.

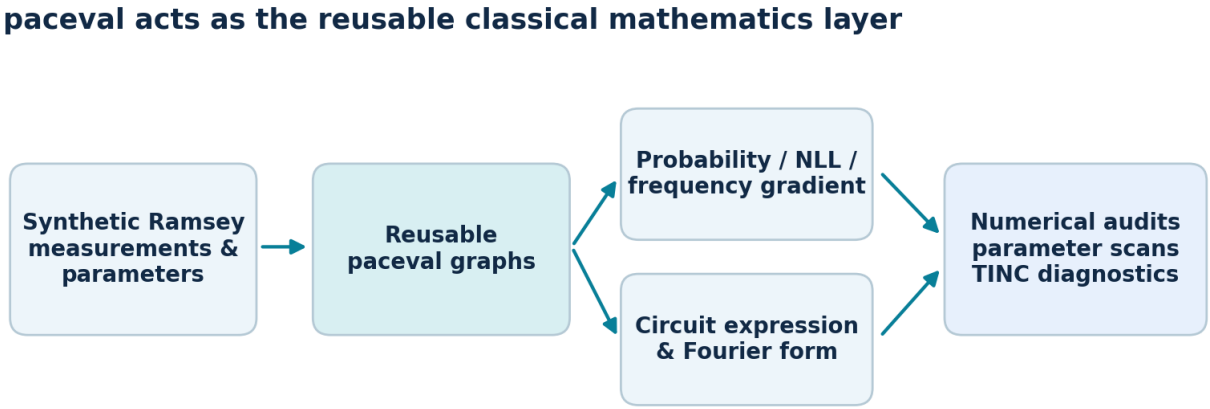


Figure 1. The implemented mathematical execution layer. The input data, formulas and numerical checks are explicit, independently reproducible components.

Feasibility questions answered by this demonstrator

1. Can the selected mathematical formulas be compiled into reusable paceval graph handles? **Yes.**
2. Can the same graphs be evaluated on batches and parameter variations without reconstruction? **Yes.**
3. Can several Ramsey-related formulas be evaluated in one documented multi-graph call? **Yes.**
4. Can results be checked independently against analytic, statevector and high-precision references? **Yes, for the stated tests.**

The demonstration uses no real quantum-device dataset. A reusable expression is a classical numerical computation object, not a quantum gate or quantum simulation backend.

2 Use case A: Ramsey calibration mathematics

A phenomenological Ramsey model describes a damped oscillatory measurement probability. With delay t, contrast C, dephasing time τ, detuning Δf and phase φ, the implemented function is:

$$P(t) = 1/2 [1 + C \exp(-t/\tau) \cos(2\pi \Delta f\, t + \varphi)]$$

For k positive outcomes among N binomial trials at one delay, the negative log-likelihood contribution (excluding parameter-independent combinatorial factors) is:

$$\ell(t) = -[k \ln P(t) + (N-k) \ln(1-P(t))]$$

The C++ implementation also computes the analytic derivative with respect to the detuning parameter:

$$\partial P/\partial(\Delta f) = -\pi\, C\, t \exp(-t/\tau) \sin(2\pi \Delta f\, t + \varphi)$$

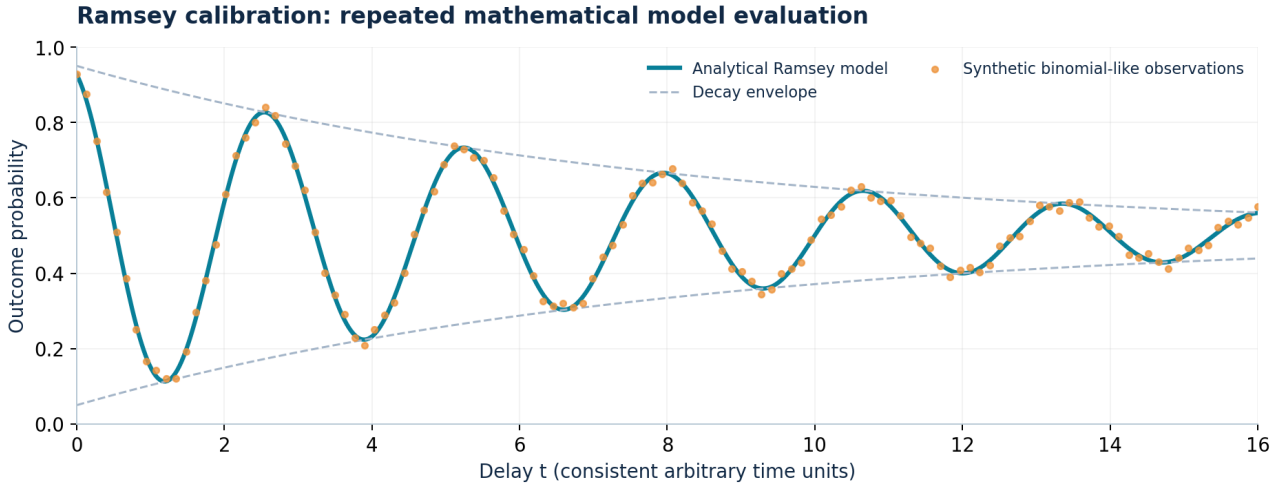


Figure 2. The analytical Ramsey curve together with seeded synthetic binomial observations; these points do not represent a laboratory measurement.

Concrete implementation and validation

The probability, likelihood contribution and frequency derivative are each represented by a persistent paceval graph. The three graphs are also evaluated together using the dedicated multi-graph API. The same likelihood graph is reused for five prescribed frequency offsets, first individually and then in one flattened input batch.

The native C++ checks cover probability and likelihood domains, finite values and a centered finite-difference test of the analytical frequency derivative. The demonstration evaluates contributions needed by an eventual fitting workflow, but **it does not execute an end-to-end maximum-likelihood fit or estimate real-device calibration parameters**.

3 Use case B: an explicit quantum-circuit function

The reference circuit begins in |00⟩, applies R_y(θ1) to its first qubit and R_y(θ2) to its second qubit, then applies CNOT with qubit 1 as control and qubit 2 as target. The measured observable is I ⊗ Z. The exact expectation value has the following equivalent forms:

f(θ1,θ2) = cos(θ1) cos(θ2)

f(θ1,θ2) = [cos(θ1 - θ2) + cos(θ1 + θ2)] / 2

The first expression is a compact product; the second is an explicit finite trigonometric (Fourier) form. The C++ program constructs one paceval graph for each form and evaluates both with independent test inputs.

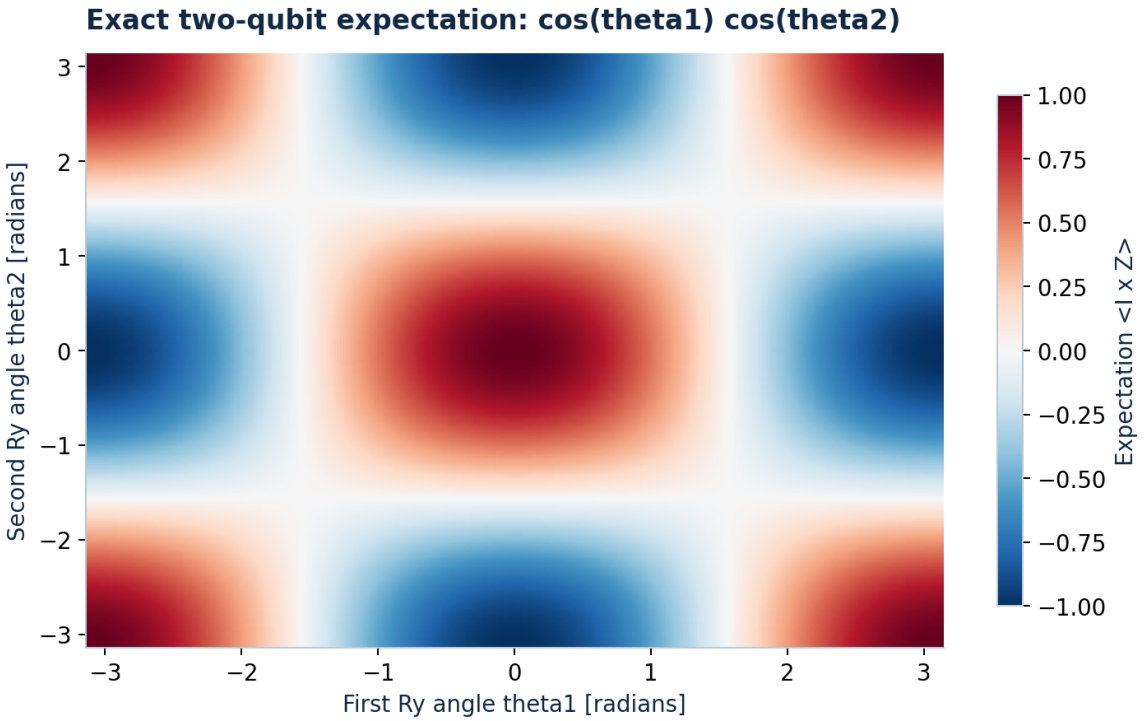


Figure 3. Analytical expectation value across two independent rotation angles, with values bounded by -1 and +1.

Independent physical cross-check

A separate C++ routine constructs the four real amplitudes from the two rotation angles, applies the CNOT permutation, and evaluates the I ⊗ Z observable from the output state. This calculation does not call paceval and checks the circuit interpretation independently of the trigonometric identity.

Only a **known, exact, compact expression** is evaluated. Fourier coefficient extraction, general statevector simulation and efficiency for arbitrary quantum circuits are outside the implemented scope.

4 Specific paceval implementation

The C++17 demonstrator uses the original paceval API version 4.25 header/source pair. The application includes paceval_main.h; src/paceval_main.cpp is compiled alongside src/quantum.cpp and forwards declared API functions to the original externally linked Linux shared library. The shared object itself is not included. There is no manually resolved function-pointer ABI or substitute engine.

paceval API entry point	Function exercised in this project
paceval_CreateComputation	Creates the five persistent expression graphs and the two TINC-specific graphs
paceval_dGetComputationResultExt	Evaluates a graph against arrays of inputs; used for Ramsey, likelihood, derivative, circuit and frequency studies
paceval_dGetMultipleComputationsResultsExt	Evaluates the three Ramsey graph handles against the same batch
paceval_dGetComputationResult	Retrieves individual TINC outputs: value, lower endpoint and upper endpoint
paceval_GetIsError and paceval_GetErrorInformation	Verifies graph initialization and evaluation state; rejects reported errors
paceval_InitializeLibrary, paceval_DeleteComputation, paceval_FreeLibrary	Initializes the runtime and releases graph and library resources

Implemented graph inventory

G1: Ramsey probability P(t). **G2:** Binomial negative log-likelihood contribution $\ell(t)$. **G3:** Frequency derivative $\partial P/\partial \Delta f$. **G4:** Quantum-circuit expectation $\cos(\theta 1)\cos(\theta 2)$. **G5:** Equivalent explicit trigonometric expression.

T1/T2: Two further computation handles with TINC enabled examine point intervals for the Ramsey probability and compact circuit expectation. The same mathematical expressions are also evaluated independently outside paceval.

All graphs declare the same ordered nine variables. Batch data is event-major and multi-graph output is graph-major. API flags, graph errors and per-event error codes are checked before comparison with independent native reference computations. The C++ RAII wrapper releases all seven handles before the paceval library is shut down. The supplied SDK source retains its original AGPL notices; it was converted to UTF-8 and only its local header include was adapted.

5 Independent numerical checks and TINC

The implementation includes a native mathematical reference, an independent statevector evaluator, a derivative finite-difference check, and an additional 110-decimal-digit Python calculation of the TINC test expressions. The Python reference promotes the actual input binary64 values and the rounded graph constant used for 2π before evaluating the expression.

Verification task	Observed outcome
Native mathematical and circuit checks	C++ self-test completed; 2,500 deterministic circuit/reference samples tested
paceval numerical evaluation	Five graphs and the multi-graph path completed with the original library; the configured mismatch tolerance checks passed
Five detuning-frequency studies	The same likelihood graph handled all five specified parameter settings
TINC point checks	24 / 24 reference values enclosed the returned lower/upper point endpoints, assessed with 110-decimal-digit mpmath arithmetic

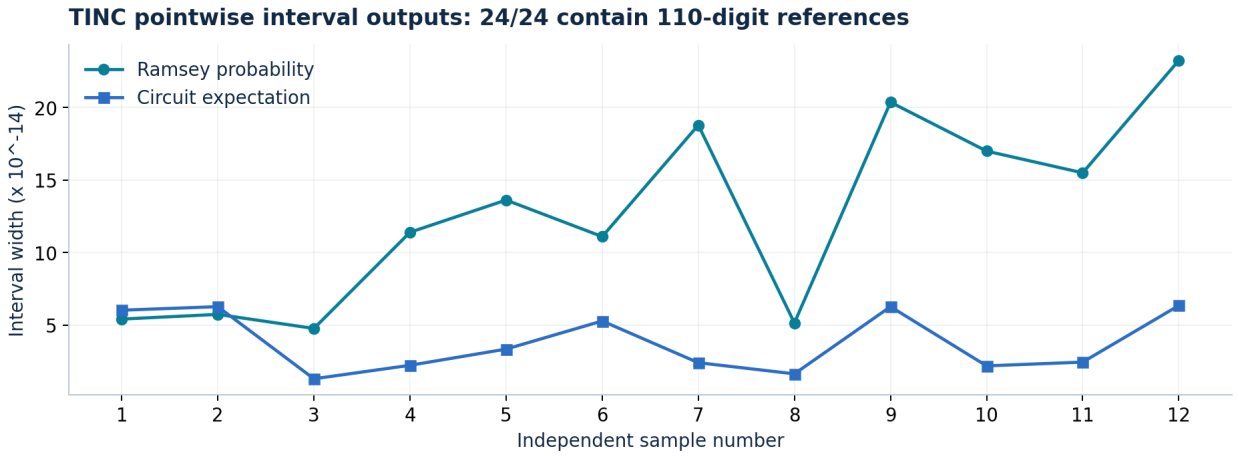


Figure 4. Widths of the actual TINC point-interval outputs for twelve Ramsey and twelve circuit cases. All 24 independent high-precision references lay inside the reported bounds.

Scientific interpretation

These successful point checks are meaningful for the **tested inputs and expression definitions**. They do not establish a proof that the TINC implementation returns rigorous enclosures for every possible input or expression. Statistical measurement noise, model mismatch and floating-point numerical uncertainty remain distinct categories of uncertainty.

6 Reproduction and included research material

This project was tested on 10 November 2026 using Debian GNU/Linux 13 (x86-64), GCC 14.2, Clang 17 and C++17, with a developer-supplied original paceval Linux shared library. Its original version 4.25 SDK source adapter was compiled and linked into the application. No relative speed comparisons are presented or inferred.

Original shared-library SHA-256:
acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12

Build from the repository root

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so

cmake -S src -B build -DCMAKE_BUILD_TYPE=Release

cmake --build build -j

ctest --test-dir build --output-on-failure
```

Execute included example data

```
./build/quantum --input-csv examples/ramsey_circuit_sample.csv --events 120 --batch 32 --csv example_results.csv

python3 src/high_precision_reference.py tinc_samples.csv
```

The example writes a separate TINC CSV next to the output file. The high-precision check needs Python 3 with mpmath. The original runtime is not bundled, but it is required during CMake configuration, linking and execution; there is no silent native-only fallback. Output CSV timing columns are engineering diagnostics only, not a research claim about relative execution speed.

Repository files

Repository path	Purpose
README.md	Use cases, verified outcomes, figures and commands
paceval_Quantum_Research_Whitepaper_EN.pdf	This research report
src/quantum.cpp; src/CMakeLists.txt	C++ demonstration and tested Linux build configuration
src/paceval_main.h; src/paceval_main.cpp	Original v4.25 API declarations and forwarding source (AGPL)
src/high_precision_reference.py	Independent 110-digit TINC point-containment check
examples/ramsey_circuit_sample.csv; examples/tinc_samples.csv	Deterministic example inputs and archived TINC point outputs
figures/00_quantum_research_hero.png	Marketing hero for the README
figures/01_ramsey_calibration.png; figures/02_quantum_circuit_fourier.png	Analytical research-use-case illustrations
figures/03_execution_graph_architecture.png; figures/04_tinc_validation.png	Architecture and numerical checks

7 Scientific findings and next research steps

The investigated use case has been successfully implemented. The original paceval engine, accessed through the supplied and compiled SDK source pair, created reusable graphs for both the Ramsey and explicit quantum-circuit functions, evaluated batches of changing input values, executed multiple Ramsey graphs together, and reused an existing likelihood expression across five frequency settings. The evaluated outputs agreed with independently constructed numerical and analytical checks within the configured tolerances.

The TINC extension produced pointwise intervals for the selected formulas. All 24 archived test cases enclosed independently computed 110-decimal-digit reference values. This provides a reproducible, bounded finding about the sampled numerical outputs; further mathematical and implementation-wide analysis would be required to extend it to rigorous, general interval guarantees.

Scientific relevance

A stable library boundary for explicitly defined classical functions is useful when mathematical models are shared across calibration studies, parameter investigations, and scientific verification workflows. The demonstrated API functionality is a concrete basis for future integration experiments with established quantum research toolchains.

Possible follow-on research

Further work could connect the same numerical kernels to documented measurements from a real device, wrap them in a complete parameter-estimation procedure with statistical uncertainty analysis, test additional explicit Fourier models, and extend pointwise numerical checks to independently verified domain enclosures. These tasks have not been claimed as completed in the present demonstrator.

Research references

[1] Qiskit Experiments documentation: T2* Ramsey characterization. https://qiskit-community.github.io/qiskit-experiments/stable/0.8/manuals/characterization/t2ramsey.html

[2] PennyLane documentation: Fourier analysis and the frequency spectrum of parameterized circuits. https://docs.pennylane.ai/en/stable/code/qp_fourier.html

[3] paceval UG: mathematical engine, reusable computation API, and TINC vendor documentation. https://paceval.com/

[4] GNU MPFR and mpmath project documentation, for independent high-precision numerical checking. https://mpmath.org/

This is an independent paceval UG research demonstrator and does not imply a collaboration with the above research software projects. The conclusions apply exclusively to the stated mathematical models and validation experiments.